

代码解释

一、TINY词法规则：

- 关键字：if, then, else, end, repeat, until, read, write
- 标识符：ID = letter letter* (字母开头，后续为字母)
- 数字：NUM = digit digit*
- 空白符：空格、换行\n、制表符\t (跳过)
- 注释：{...} 花括号包围，不可嵌套
- 专用符号：+, -, *, /, <, =, (,), :=

二、DFA状态机设计

```
1 stateDiagram-v2
2     [*] --> START
3
4     START --> INNUM : digit
5     START --> INID : letter
6     START --> INASSIGN : =
7     START --> INCOMMENT : {
8     START --> START : whitespace
9     START --> DONE : 其他符号
10
11     INNUM --> INNUM : digit
12     INNUM --> DONE : other (回退)
13
14     INID --> INID : letter
15     INID --> DONE : other (回退)
16
17     INASSIGN --> DONE : =
18     INASSIGN --> DONE : other (错误)
19
20     INCOMMENT --> INCOMMENT : other
21     INCOMMENT --> START : }
22
23     DONE --> [*]
```

5个DFA状态：

状态	含义	转移条件
START	起始状态	根据首字符分发
INNUM	读取数字	digit→继续，其他→完成NUM
INID	读取标识符	letter→继续，其他→完成ID
INASSIGN	读取赋值号	= →完成ASSIGN，其他→错误

状态	含义	转移条件
INCOMMENT	在注释中	} → 返回START, 其他 → 继续
DONE	完成识别	返回Token

三、代码结构

3.1 Token类型枚举（第15-35行）

```
1  typedef enum {
2      /* 保留字 */
3      IF, THEN, ELSE, END, REPEAT, UNTIL, READ, WRITE,
4      /* 标识符和数字 */
5      ID, NUM,
6      /* 专用符号 */
7      PLUS, MINUS, TIMES, OVER, LT, EQ, LPAREN, RPAREN, ASSIGN, SEMI,
8      /* 特殊标记 */
9      ENDFILE, ERROR
10 } TokenType;
```

3.2 DFA状态枚举（第38-45行）

```
1  typedef enum {
2      START,      // 起始
3      INNUM,      // 读数字
4      INID,       // 读标识符
5      INASSIGN,   // 读:=
6      INCOMMENT,  // 读注释
7      DONE       // 完成
8  } StateType;
```

3.3 保留字查找表（第75-98行）

```
1  static ReservedWord reservedWords[] = {
2      {"if", IF}, {"then", THEN}, {"else", ELSE}, {"end", END},
3      {"repeat", REPEAT}, {"until", UNTIL}, {"read", READ}, {"write", WRITE}
4  };
5
6  // 查找函数：匹配返回关键字类型，否则返回ID
7  static TokenType reservedLookup(char* s) {
8      for (int i = 0; i < RESERVED_WORDS_COUNT; i++) {
9          if (strcmp(s, reservedWords[i].str) == 0)
10             return reservedWords[i].tok;
11     }
12     return ID;
13 }
```

3.4 核心函数 getToken() (第165-320行)

主循环逻辑:

```
1  TokenType getToken(void) {
2      StateType state = START;
3      while (state != DONE) {
4          int c = getNextChar();
5          switch (state) {
6              case START:
7                  if (isdigit(c)) state = INNUM;
8                  else if (isalpha(c)) state = INID;
9                  else if (c == ':') state = INASSIGN;
10                 else if (c == '{') state = INCOMMENT;
11                 else if (isspace(c)) /* 跳过 */;
12                 else /* 单字符符号或错误 */;
13                 break;
14             case INNUM:
15                 if (!isdigit(c)) { ungetNextChar(); state = DONE;
currentToken = NUM; }
16                 break;
17             case INID:
18                 if (!isalpha(c)) { ungetNextChar(); state = DONE;
currentToken = ID; }
19                 break;
20             case INASSIGN:
21                 if (c == '=') currentToken = ASSIGN;
22                 else currentToken = ERROR;
23                 state = DONE;
24                 break;
25             case INCOMMENT:
26                 if (c == '}') state = START; // 注释结束, 回到起点
27                 break;
28         }
29     }
30     // 标识符完成后检查是否为保留字
31     if (currentToken == ID)
32         currentToken = reservedLookup(tokenString);
33     return currentToken;
34 }
```

关键点:

1. `ungetNextChar()` 回退字符 - 当发现非数字/字母时, 该字符属于下一个Token
 2. 注释处理 - `INCOMMENT` 状态下遇到 `}` 才返回 `START`, 内容全部丢弃
 3. 保留字识别 - 先按ID读取, 完成后查表判断是否为关键字
-

四、运行

编译

```
1 | gcc -o tiny_lexer tiny_lexer.c
```

测试程序 sample.tny

```
1 | { 注释会被跳过 }
2 | read x;
3 | if x < 0 then
4 |     x := 0 - x
5 | end;
6 | write x
```

运行

```
1 | ./tiny_lexer sample.tny
```

输出示例

```
1 |      4: read x;
2 |          4: READ
3 |          4: ID      , val= x
4 |          4: SEMI
5 |      5: if x < 0 then
6 |          5: IF
7 |          5: ID      , val= x
8 |          5: LT
9 |          5: NUM      , val= 0
10 |         5: THEN
```

附：文件清单

文件	说明
<code>tiny_lexer.c</code>	词法分析器源代码
<code>sample.tny</code>	测试用TINY程序
<code>代码讲解.md</code>	本文档